

Claude Code — Full Reference

A detailed, working reference for the Claude Code CLI — install, every subcommand and flag, every slash command, permissions, settings, memory, MCP, and plugins. Companion to `ai-agent-concepts.md`, which covers the generic ideas (agentic loop, context windows, permission models) that apply to any AI agent and aren't repeated here.

Written to be exported to PDF (VS Code + the `yzane.markdown-pdf` extension: open this file, `Ctrl+Shift+P` → **Markdown PDF: Export (pdf)**). Every link below points at `code.claude.com/docs/en/...`, verified live against the site on 2026-09-11, not recalled from memory — this moves faster than most docs, and a stale cached URL is worse than no link. Refined over time rather than written once and frozen.

Official docs hub: code.claude.com/docs/en/overview

What it is, and where it runs

Claude Code is Anthropic's agentic coding tool: reads your codebase, edits files, runs commands, integrates with git and your dev tools. The same engine runs on several **surfaces**, all sharing the same `CLAUDE.md` files, settings, and MCP servers:

Surface	What it adds
Terminal CLI	Full-featured, scriptable — this document's focus
VS Code / JetBrains	Inline diffs, @-mentions, plan review in the editor
Desktop app	Visual diff review, multiple sessions side by side, scheduling
Web (<code>claude.ai/code</code>)	No local setup, long-running tasks, work on repos you don't have locally

Install

```
# macOS, Linux, WSL
curl -fsSL https://claude.ai/install.sh | bash

# Windows PowerShell
irm https://claude.ai/install.ps1 | iex

# Windows CMD
curl -fsSL https://claude.ai/install.cmd -o install.cmd && install.cmd && del install.cmd
```

Also installable via `brew install --cask claude-code` (Homebrew), `winget install Anthropic.ClaudeCode` (WinGet), or `apt/dnf/apk` on Debian/Fedora/RHEL/Alpine. **Native installs auto-update in the background**; Homebrew/WinGet installs need a manual `brew upgrade claude-code / winget upgrade Anthropic.ClaudeCode`.

```
cd your-project
claude          # first run prompts you to log in
```

Docs: [setup](#) (advanced install, manual updates, uninstall) · [troubleshoot-install](#) · [quickstart](#)

CLI subcommands

Everything after `claude` on the command line. The bare `claude` command starts an interactive session; everything else here is a distinct subcommand.

Command	Does
<code>claude</code>	Start interactive session
<code>claude "query"</code>	Start interactive session with an opening prompt
<code>claude -p "query"</code>	Print mode: answer once, then exit (scriptable)
<code>claude -c</code>	Continue the most recent conversation in this directory
<code>claude -r "<session>"</code>	Resume a session by ID or name
<code>claude update</code>	Update to the latest version
<code>claude install [version]</code>	Install or reinstall the native binary
<code>claude auth login / logout / status</code>	Sign in / out / show auth status
<code>claude setup-token</code>	Generate a long-lived OAuth token for CI/scripts
<code>claude doctor</code>	Read-only installation and settings diagnostics
<code>claude mcp ...</code>	Configure MCP servers (see MCP below)
<code>claude plugin ...</code>	Manage plugins (see Plugins below)
<code>claude agents</code>	Open agent view: monitor/dispatch parallel background sessions
<code>claude attach <id></code>	Attach to a background session in this terminal
<code>claude respawn <id></code>	Restart a background session, conversation intact
<code>claude logs <id></code>	Print recent output from a background session
<code>claude stop <id> (alias kill)</code>	Stop a background session
<code>claude rm <id></code>	Remove a background session from the list
<code>claude daemon status / stop</code>	Background-session supervisor status / stop it
<code>claude gateway</code>	Start the self-hosted Claude apps gateway server
<code>claude self-hosted-runner</code>	Start a runner process for self-hosted environments

Command	Does
<code>claude remote-control</code>	Start Remote Control server (control from claude.ai)
<code>claude import [source]</code>	Bring configuration in from another coding agent
<code>claude project purge [path]</code>	Delete all local Claude Code state for a project
<code>claude auto-mode defaults</code>	Print built-in auto-mode classifier rules as JSON
<code>claude auto-mode reset</code>	Restore default auto-mode configuration
<code>claude ultrareview [target]</code>	Run ultrareview non-interactively

CLI flags

The practical set, grouped. There are more (see the full [cli-reference](#)) but these cover normal day-to-day and scripted use.

Session basics

Flag	Purpose
<code>-p, --print</code>	Answer once without interactive mode
<code>-c, --continue</code>	Load the most recent conversation here
<code>-r, --resume "<session>"</code>	Resume a specific session by ID or name
<code>-n, --name</code>	Name the session, for easier resuming later
<code>--fork-session</code>	Resume into a <i>new</i> session ID instead of reusing the original
<code>--add-dir</code>	Grant access to an extra directory outside the cwd
<code>--bg, --background</code>	Start as a background agent, return immediately

Model & behavior

Flag	Purpose
<code>--model</code>	Pick a model (alias or full name) for this session
<code>--effort</code>	<code>low</code> / <code>medium</code> / <code>high</code> / <code>xhigh</code> / <code>max</code> / <code>ultracode</code>
<code>--fallback-model</code>	Auto-switch model if the primary is overloaded
<code>--max-turns</code>	Cap the number of agentic-loop iterations
<code>--max-budget-usd</code>	Hard dollar cap on API spend for the session
<code>--agent</code>	Use a specific (sub)agent definition for the main thread
<code>--agents</code>	Define custom subagents dynamically via JSON

Permissions & tools

Flag	Purpose
<code>--permission-mode</code>	Start in a specific mode — see Permission modes
<code>--dangerously-skip-permissions</code>	Shorthand for <code>--permission-mode bypassPermissions</code>
<code>--allowedTools</code> / <code>--disallowedTools</code>	Allow/deny specific tools without prompting
<code>--permission-prompt-tool</code>	MCP tool that answers permission prompts in non-interactive mode

MCP & plugins

Flag	Purpose
<code>--mcp-config</code>	Load MCP servers from JSON files or an inline string
<code>--plugin-dir</code>	Load a plugin from a local directory or <code>.zip</code> (repeatable)
<code>--plugin-url</code>	Fetch a plugin <code>.zip</code> from a URL for this session only

System prompt

Flag	Purpose
<code>--append-system-prompt</code> / <code>--append-system-prompt-file</code>	Add to the default system prompt
<code>--system-prompt</code> / <code>--system-prompt-file</code>	Replace the entire system prompt
<code>--append-subagent-system-prompt[-file]</code>	Same, but for every subagent's prompt

Scripting & automation

Flag	Purpose
<code>--output-format</code>	<code>text</code> / <code>json</code> / <code>stream-json</code>
<code>--input-format</code>	<code>text</code> / <code>stream-json</code>
<code>--json-schema</code>	Get validated JSON output matching a JSON Schema
<code>--exec</code>	Run a shell command as a PTY-backed background job
<code>--no-session-persistence</code>	Don't write a session transcript at all
<code>--autocompact <auto tokens></code>	Set the auto-compact window for this session

Diagnostics

Flag	Purpose
<code>--debug</code>	Enable debug mode (optionally filtered by category)

Flag	Purpose
<code>--debug-file <path></code>	Write debug logs to a specific file
<code>--safe-mode</code>	Start with all customizations disabled, for troubleshooting
<code>--bare</code>	Minimal mode — skip auto-discovery for faster startup

Full reference (every flag, including web/remote/CI-specific ones not listed above): [cli-reference](#)

Piping — Claude Code follows the Unix philosophy both directions:

```
tail -200 app.log | claude -p "flag anything unusual"
git diff main --name-only | claude -p "review these files for security issues"
```

Slash commands

Typed inside a running session. Run `/help` any time for the current, authoritative list — it changes across versions faster than anything else in this document. Organized here by what they're for.

Project setup & configuration

Command	Does
<code>/init</code>	Generate a starting <code>CLAUDE.md</code> by analyzing the repo
<code>/memory</code>	Edit <code>CLAUDE.md</code> files, toggle auto memory, view memory entries
<code>/mcp</code> <code>[reconnect enable disable]</code>	Manage MCP server connections and OAuth
<code>/permissions</code>	View/edit allow, ask, and deny rules
<code>/config [key=value]</code>	Open settings, or set a key directly
<code>/keybindings</code>	Open the keyboard-shortcuts file
<code>/hooks</code>	View hook configuration for tool events

Model & performance

Command	Does
<code>/model [model]</code>	Switch model, saves as default
<code>/effort [level auto status]</code>	Set effort level
<code>/fast [on off]</code>	Toggle fast mode
<code>/autocompact [auto tokens]</code>	Set the auto-compact window

Command	Does
<code>/compact [instructions]</code>	Manually free context by summarizing
<code>/advisor [model off]</code>	Enable/disable the server-side advisor tool

Navigation & session management

Command	Does
<code>/cd <path></code>	Move the session to a new working directory
<code>/add-dir <path></code>	Add a working directory for file access
<code>/clear [name]</code>	Start a new conversation, empty context
<code>/resume</code>	Interactive picker for an earlier conversation
<code>/branch [name]</code>	Branch the current conversation
<code>/fork [prompt]</code>	Copy the current conversation into a new background session
<code>/background [prompt] (alias <code>/bg</code>)</code>	Detach the session to run in the background
<code>/teleport</code>	Pull a web session into the terminal
<code>/remote-control</code>	Continue this session from another device
<code>/exit (alias <code>/quit</code>)</code>	Exit the CLI

Code review & quality

Command	Does
<code>/diff</code>	Review changes in the working tree
<code>/code-review [level] [--fix] [--comment] [pr# branch path] (alias <code>/review</code>)</code>	Review a diff/PR for bugs and cleanup
<code>/security-review</code>	Check the diff for security vulnerabilities
<code>/simplify</code>	Simplification pass (reuse/efficiency, not bug-hunting)

Development workflows

Command	Does
<code>/plan [description]</code>	Enter plan mode
<code>/batch <instruction></code>	Orchestrate large-scale changes across the codebase in parallel
<code>/debug [description]</code>	Enable debug logging, troubleshoot
<code>/goal [condition clear]</code>	Set a goal Claude works toward until met

Command	Does
<code>/tasks</code>	List the session's background work, including subagents
<code>/subtask</code>	Hand a side task to a subagent that reports back
<code>/loop [interval] [prompt]</code> (alias <code>/proactive</code>)	Run a prompt repeatedly on a schedule

Utilities & information

Command	Does
<code>/help</code>	Show help and the current command list
<code>/status</code>	Session status
<code>/usage</code> (alias <code>/cost</code>)	API usage and cost
<code>/context [all]</code>	Visualize current context usage
<code>/btw [question]</code>	Side question that doesn't enter conversation history
<code>/copy [N]</code>	Copy the last (or Nth) assistant response
<code>/export [filename]</code>	Export the conversation as plain text
<code>/theme / /color</code>	Theme / prompt-bar color
<code>/focus</code>	Toggle a focus view (last prompt/response only)

Feedback & reporting

Command	Does
<code>/feedback [report]</code>	Send product feedback
<code>/bug [report]</code> (alias <code>/share</code>)	Report a bug / share the conversation
<code>/doctor</code>	Setup checkup and diagnosis

Code recovery

Command	Does
<code>/rewind</code>	Roll code and conversation back to a checkpoint

Design

Command	Does
<code>/design [brief]</code>	Draft UI mockups as Artifacts
<code>/design-sync [hint]</code>	Convert a React design system, upload to Claude Design

Command	Does
<code>/dataviz [request]</code>	Guidance for charts, graphs, dashboards

Research

Command	Does
<code>/deep-research <question></code>	Fan out web searches, synthesize a cited report
<code>/insights</code>	HTML report analyzing recent sessions

Account

Command	Does
<code>/login / /logout</code>	Sign in / out
<code>/upgrade</code>	Upgrade plan (not shown on Enterprise)

IDE & integrations

Command	Does
<code>/ide</code>	Manage IDE integrations
<code>/desktop</code> (alias <code>/app</code>)	Continue this session in the Desktop app
<code>/mobile</code> (aliases <code>/ios</code> , <code>/android</code>)	QR code for the Claude mobile app
<code>/chrome</code>	Configure Claude-in-Chrome
<code>/plugin [subcommand]</code>	Manage plugins — see Plugins
<code>/reload-plugins</code>	Reload plugins after a change
<code>/install-github-app / /install-slack-app</code>	Install the GitHub / Slack app
<code>/autofix-pr [prompt]</code>	Spawn a session that watches a PR and pushes fixes

Agents

Command	Does
<code>/agents</code>	Manage subagent configurations
<code>/list-agents</code> (alias <code>/peers</code>)	List subagents, teammates, other sessions

Misc

Command	Does
<code>/import [codex gemini cursor]</code>	Bring in configuration from another tool

Command	Does
<code>/fewer-permission-prompts</code>	Scan transcripts, add an allowlist to reduce prompts

Full, current list: [commands](#)

Permission modes

Set via `permissions.defaultMode` in settings, or `--permission-mode` / `--dangerously-skip-permissions` on the command line.

Mode	Behavior
<code>default</code>	Prompts before anything that reads/writes outside a small safe set
<code>plan</code>	Research-only — proposes a plan, makes no changes until approved
<code>acceptEdits</code>	Auto-approves file edits, still prompts for other risky actions
<code>auto</code>	Auto-approves routine actions, still stops for destructive/irreversible ones
<code>bypassPermissions</code>	No prompts at all, for anything, ever
<code>dontAsk</code>	Alias-adjacent mode, similar intent to <code>auto</code>

Verified gotcha (Claude Code ≥ v2.1.257): `auto` and `bypassPermissions` only take effect from **user** (`~/.claude/settings.json`) or **managed** settings. Setting either in a **project** (`.claude/settings.json`) or **local** (`.claude/settings.local.json`) file is silently ignored. For project-scoped autonomy, use `--permission-mode` per invocation instead — there's no settings-file equivalent that works at that scope anymore.

Docs: [settings](#) · [permissions](#)

Settings files and precedence

Highest to lowest — a key set higher overrides the same key set lower:

1. **Managed settings** — org policy (MDM / `managed-settings.json` / claude.ai console)
2. **Command line** — `claude --settings <file>`, for one invocation
3. **Project local** — `.claude/settings.local.json` (gitignore this — personal)
4. **Shared project** — `.claude/settings.json` (commit this — team-shared)
5. **User** — `~/.claude/settings.json` (you, every project)

Full schema: [settings-reference](#)

Memory: `CLAUDE.md` vs. auto memory

Two complementary systems, both loaded at the start of every session:

	CLAUDE.md	Auto memory
Who writes it	You	Claude, on its own
Contains	Instructions, rules, conventions	Learnings, corrections, preferences
Use for	"Always run X before committing"	"This user prefers terse commit messages"

CLAUDE.md locations, broadest to most specific (all discovered files get concatenated into context, not overridden):

Scope	Location	Shared with
Managed policy	/etc/claude-code/CLAUDE.md (Linux)	Whole organization
User	~/.claude/CLAUDE.md	Just you, every project
Project	./CLAUDE.md or ~/.claude/CLAUDE.md	Team, via source control
Local	./CLAUDE.local.md	Just you, this project (gitignore it)

Claude Code reads CLAUDE.md, not AGENTS.md. If a repo already has an AGENTS.md (a cross-tool convention many agents share), the standard move is a CLAUDE.md that imports it:

```
@AGENTS.md

## Claude Code
Use plan mode for changes under src/billing/.
```

Exactly the pattern this repo's own CLAUDE.md uses. Run /init to generate a starting CLAUDE.md from the codebase, or /import to pull in configuration from Cursor/Copilot/Codex/Gemini and other tools.

Auto memory is on by default, stored at ~/.claude/projects/<project>/memory/ as a MEMORY.md index (only the first 200 lines / 25KB load automatically) plus one topic file per memory (loaded on demand). Toggle with /memory or autoMemoryEnabled in settings. Four kinds of note: user (your role/preferences), feedback (corrections you've given), project (context Claude can't derive from code), reference (pointers to external systems).

Docs: [memory](#)

MCP

```
# Remote server, HTTP transport (recommended for anything not local)
claude mcp add --transport http <name> <url>

# Local process, stdio transport
claude mcp add --transport stdio <name> -- <command> [args...]

# Scope: local (just you, this project) is default
```

```
claude mcp add --scope project --transport http <name> <url> # shared,
via .mcp.json
claude mcp add --scope user --transport http <name> <url> # all your
projects
```

Command	Does
claude mcp add	Add a server (flags above control transport/scope)
claude mcp list	List configured servers
claude mcp get <name>	Show details for one server
claude mcp remove <name>	Remove a server
claude mcp login <name> / logout <name>	OAuth sign-in / clear credentials
/mcp (in-session)	Manage servers interactively

Transports: **HTTP** (recommended for remote services), **stdio** (local processes), **SSE** (deprecated — use HTTP), **WebSocket** (JSON-config only). This document doesn't recommend specific third-party MCP servers — that's a per-task choice and the ecosystem moves fast.

Docs: [mcp](#) · [mcp-quickstart](#)

Plugins

A plugin bundles skills, agents, hooks, and/or MCP/LSP servers into one shareable, versioned unit — the step up from personal `.claude/` customization once you want to share or reuse across projects.

Using plugins someone else made:

```
/plugin marketplace add anthropics/claude-plugins-official # usually
automatic on first run
/plugin install <name>@claude-plugins-official
/plugin list [--enabled|--disabled]
/plugin enable <name>@<marketplace>
/plugin disable <name>@<marketplace>
/plugin uninstall <name>@<marketplace>
/plugin marketplace list
/plugin marketplace update <marketplace-name>
/plugin marketplace remove <marketplace-name>
```

Non-interactive equivalents for scripting (don't open the panel): `claude plugin install <name>@<marketplace> [--scope project|user|local]`, `claude plugin uninstall`, `claude plugin details <name>`. Two official Anthropic-run marketplaces exist: `claude-plugins-official` (curated, added automatically) and `claude-community` (public submissions, add manually with `/plugin marketplace add anthropics/claude-plugins-community`).

Building your own (brief — full guide linked below):

```
claude plugin init my-tool      # scaffold ~/.claude/skills/my-tool/,
loads automatically
claude --plugin-dir ./my-plugin # test a plugin without installing it
claude plugin validate ./my-plugin
```

A plugin is a directory with a `.claude-plugin/plugin.json` manifest at its root, plus any of: `skills/`, `commands/`, `agents/`, `hooks/` (`hooks.json`), `.mcp.json`, `.lsp.json`, `monitors/`, `bin/`, `settings.json`. Skills are namespaced as `/plugin-name:skill-name` to avoid collisions.

Security note: plugins and marketplaces run with your user privileges. Only install/add ones you trust — Anthropic doesn't vet third-party marketplace contents beyond automated screening.

Docs: [discover-plugins](#) (installing/managing) · [plugins](#) (building) · [plugin-marketplaces](#) (distributing) · [plugins-reference](#) (full technical spec)

Skills, hooks, and sub-agents

- **Skills** — package a repeatable workflow (e.g. "review a PR my way") that loads on demand instead of every session. Docs: [skills](#)
- **Hooks** — shell commands that run at fixed lifecycle points (before/after a tool call, on session start, etc.) — actual enforcement, unlike `CLAUDE.md` which is context Claude tries to follow. Docs: [hooks](#)
- **Sub-agents** — delegate a narrow task to a fresh agent with its own context; the main conversation only sees the result. Docs: [sub-agents](#)

Beyond the terminal

Want to...	Use
Keep working from your phone	Remote Control
Run on a schedule (cloud)	Routines
Run on a schedule (local, one session)	<code>/loop</code>
Automate PR review in CI	GitHub Actions
Build a fully custom agent on Claude Code's engine	Agent SDK

Where to go deeper

- [quickstart](#) — a full first real task, start to finish
- [common-workflows](#) and [best-practices](#)
- [troubleshooting](#)
- [glossary](#) — precise definitions for terms this document uses loosely